Data Structures

Linked Lists

Stack Implementation

- Last week's implementation
  - Built using dynamic array

Void pushStack(struct dynArr *s, int val);

int popStack(struct dynArr *s);

Int peekStack(struct dynArr *s);

TYPE

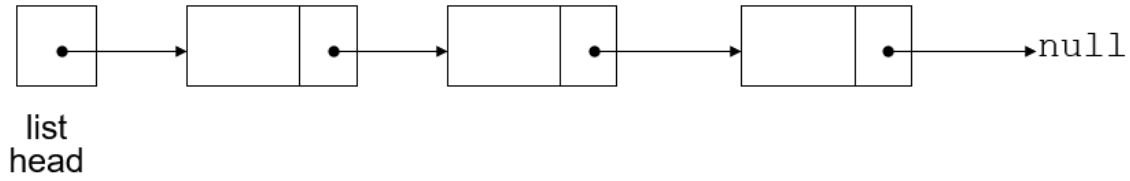
- Used to create more adaptable code
- TYPE is typically defined in a header file

- # ifndef TYPE
- # define TYPE int
- # endif

- void initList(struct List *lst);
- void addFrontList(struct Link *head, TYPE val);
- void addBackList(struct Link *head, TYPE val);

What is a Linked List?

- Alternative way to store data
- Dynamically allocated data structures can be linked together in memory to form a chain
- A linked list contains zero or more nodes
- A linked list is a series of connected nodes
- Linked lists can grow and shrink as the program runs
- Memory is allocated and freed as needed



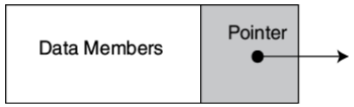
- Typical operations
  - Insert
  - Search
  - Remove

Nodes

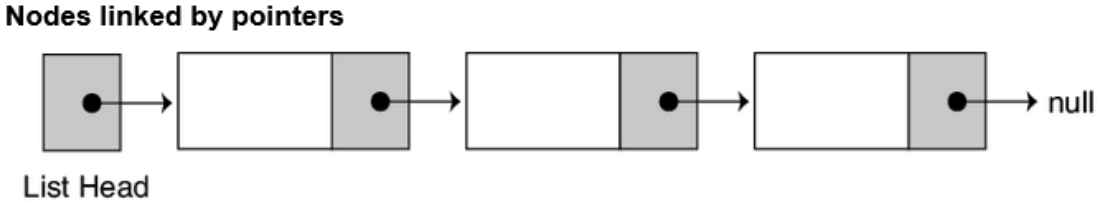
Each node in a linked list contains one or more members that contain data

Names, numbers, animals, etc

In addition to data nodes hold pointers which can point to another node



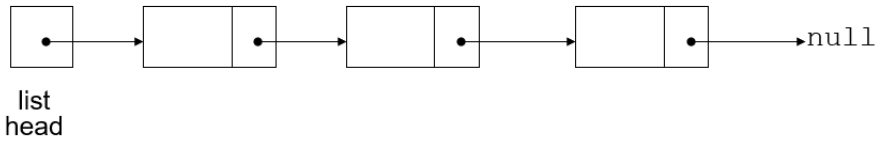
Linked lists are called linked lists because of the links between nodes implemented with pointers



Linked List Vocabulary

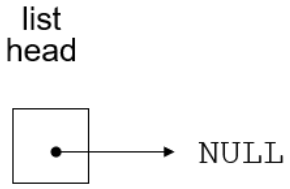
The list head
- The list head points to the first node in the list

Last node
- Contains a value
- Points to null
- Used to detect the end of the list



Nodes
- Every other node in the linked list
- Contains a value and a pointer to the next link

Empty list
- Contains zero nodes
- Head points to null



Linked List ADT

- Basic operations
  - Append a node to the end of the list
  - Insert a node within the list
  - Traverse the list
  - Delete a node
  - Destroy the list
- Head Pointer
  - Points to the first node in the linked list
  - Initialize to null
- Nodes
  - Each node points to the next node
  - The last node points to null

Linked List Advantages Over Dynamic Arrays

Linked lists can easily grow and shrink in size

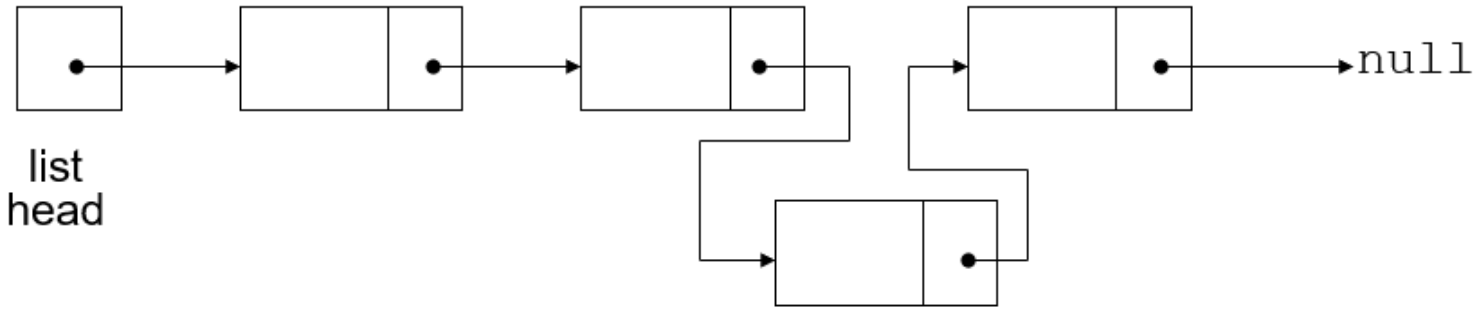
Size of the linked list is not known or needed

- Memory is allocated and freed as needed

Insert and delete only affects neighboring links

- In an array the entire array potentially shifts

Linked lists can insert a node between other nodes easily



Building a Linked List – Node Struct

You need a structure to be used with nodes

Declare a node

```
struct ListNode {
  TYPE val;
  ListNode *next;
}
```

val is the value contained in the node

Next contains the address of the next node in the list

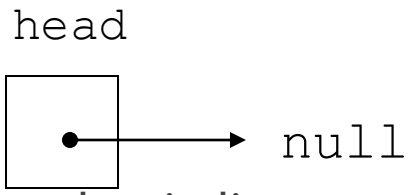
- Self-referential data structure
  - Its structure contains a pointer to an object of the same type

Building a Linked List – Init Head

Define a pointer for the head of the list:

```
ListNode *head = nullptr;
```

Head pointer initialized to `nullptr` to indicate an empty list



`Null` is used to indicate end-of-list

Should always be tested for before using a pointer

```
while (head != nullptr)
```

Building a Linked List - Operations

After the head pointer is created you need to implement operations

- Add, delete, print, search, etc

Create a New Node

Allocate memory for the new node:

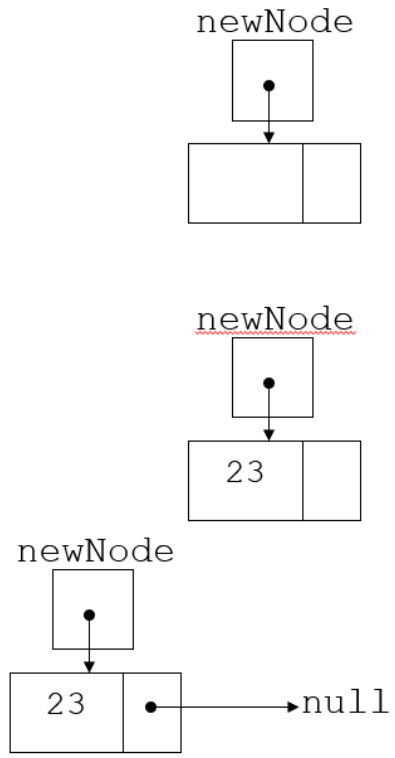
```
ListNode *newNode = malloc(sizeof(ListNode));
```

Initialize the contents of the node:

```
newNode->value = num;
```

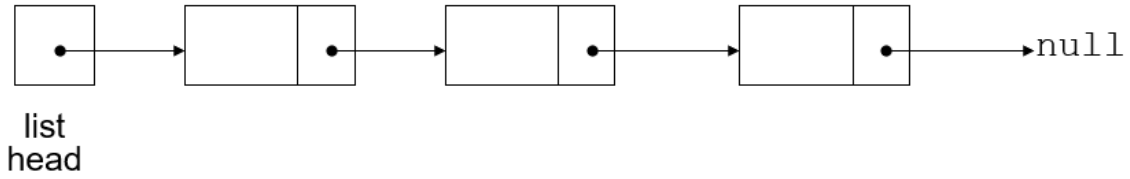
Set the pointer field to `nullptr`:

```
newNode->next = nullptr;
```



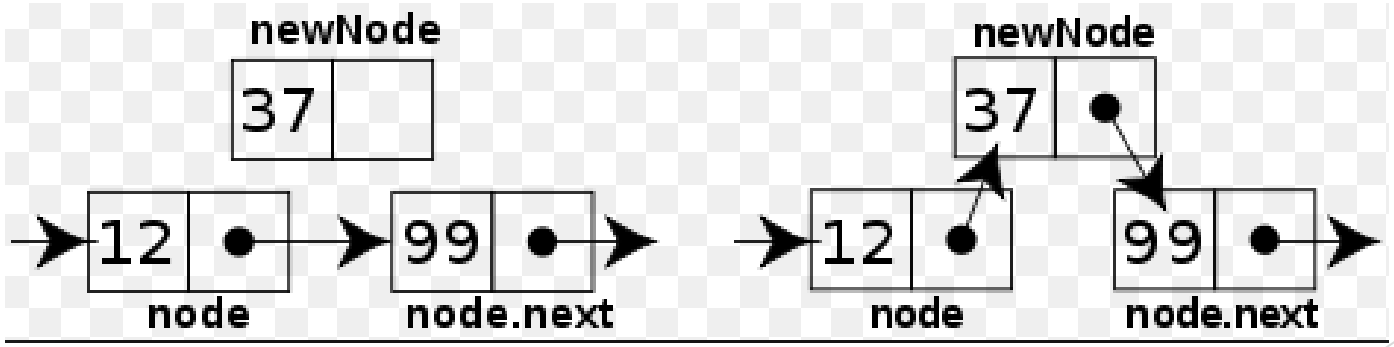
Traversing a List

- Start at the head
  - Follow pointers until we find a null pointer



Add to LinkedList

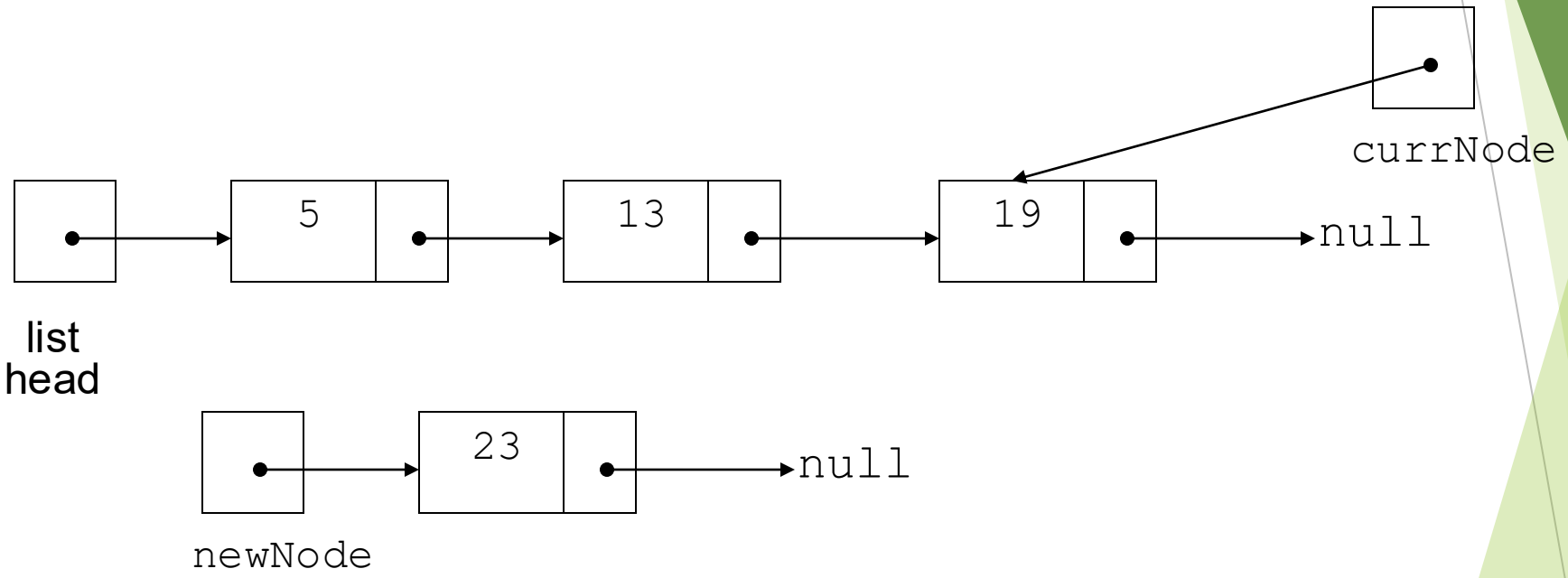
- Memory allocated
- Node initialized
- Pointers adjusted
- Sorted list vs unsorted list



Append a Node

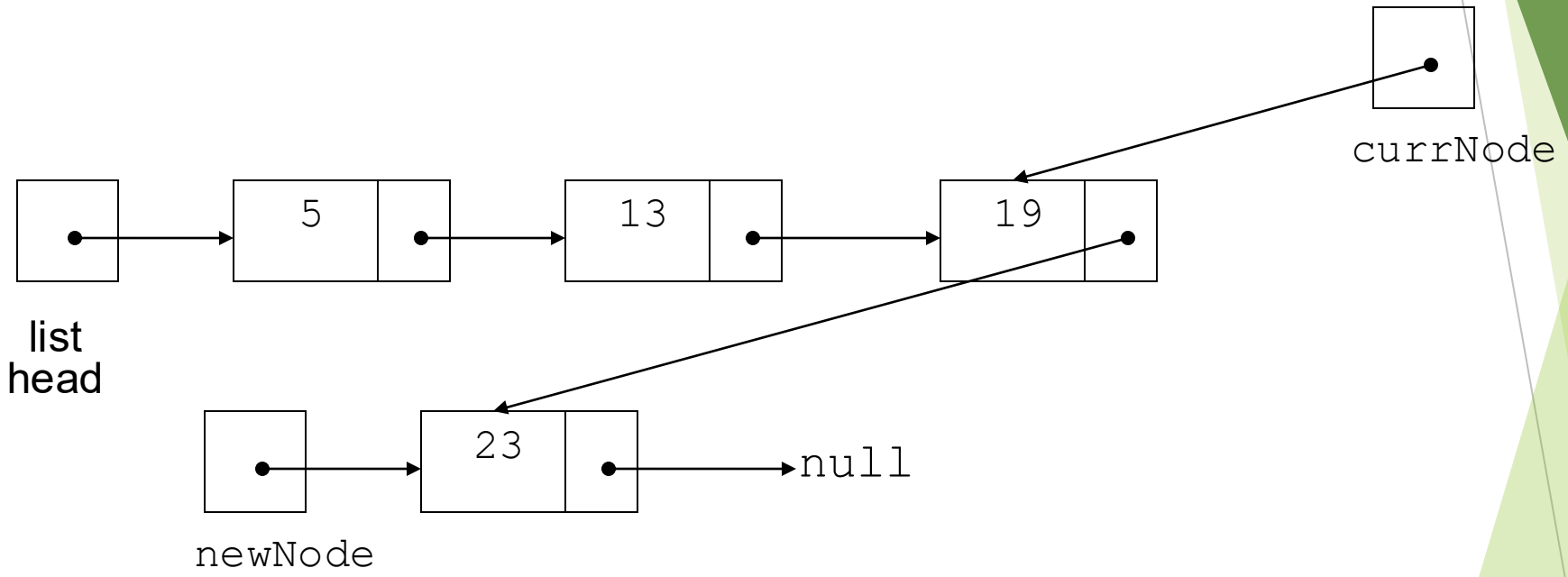
- Add a node to the end of the list
  - Create a new node
  - If the list is empty
    - Make this node the first node
    - Set head pointer to this node
  - If not empty
    - Traverse the list to the end
    - Set pointer of last node to point to new node
    - The new node will point to null

Appending a Node



New node created, end of list located

Appending a Node

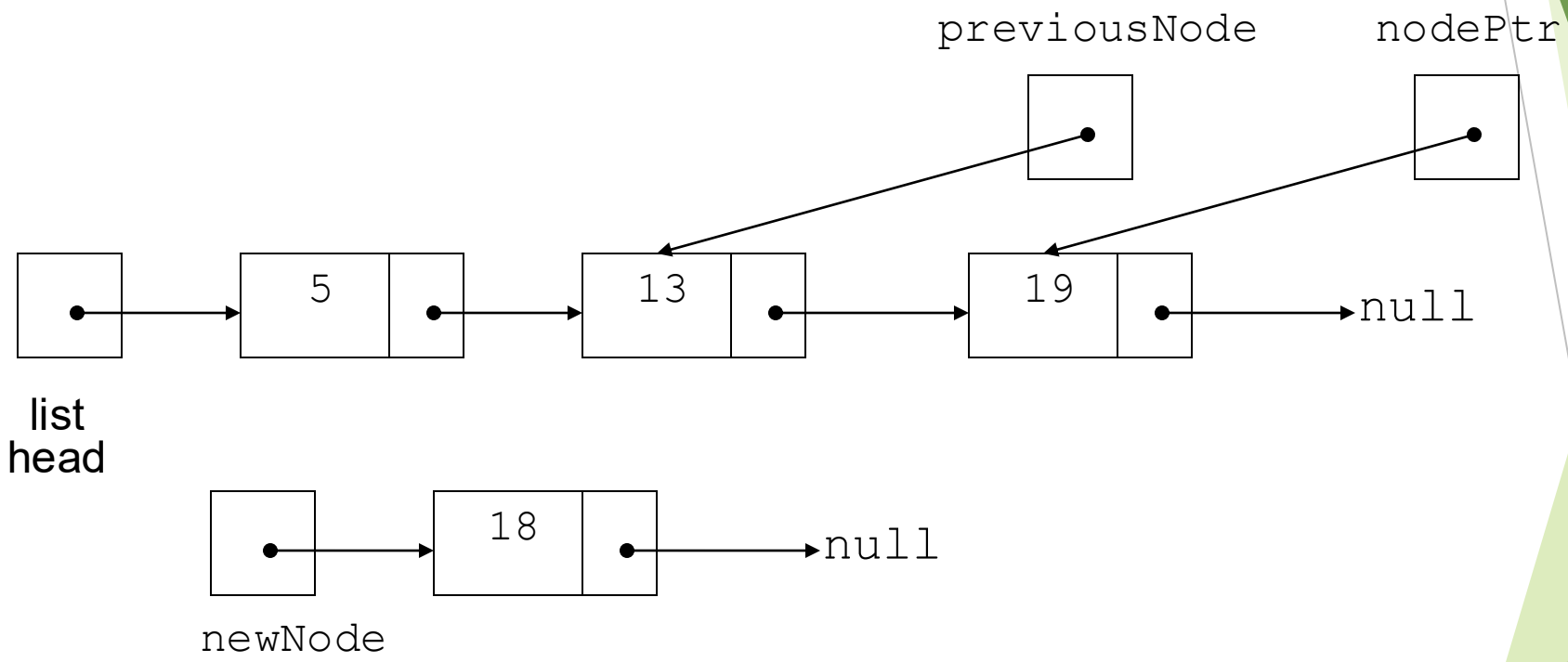


New node added to end of list

Insert

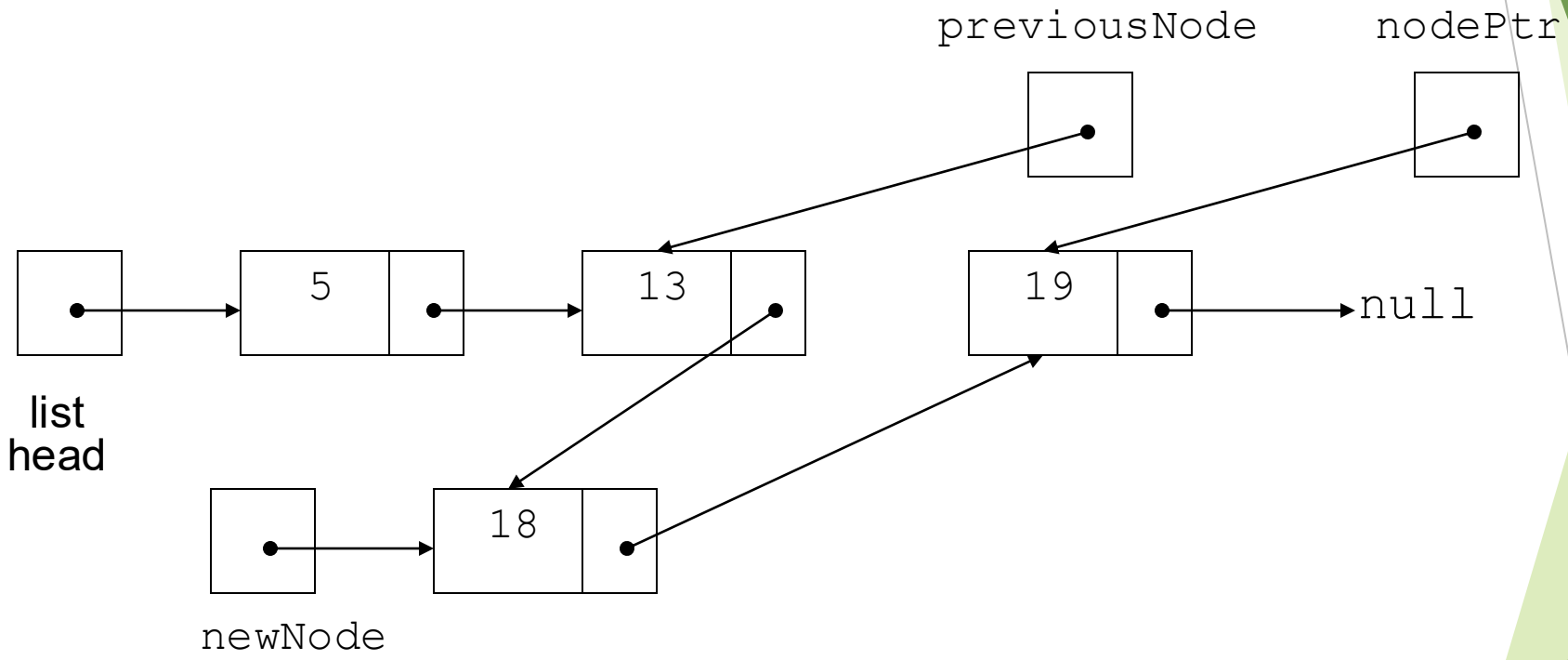
- Append is less complex than inserting into the middle of a linked list
  - Insert implies some kind of sorted order in the list
- Insert requires two pointers to traverse the list
  - Pointer to locate the node with data value greater than that of node to be inserted
  - Pointer to 'trail behind' one node, to point to node before point of insertion
- New node is inserted between the nodes pointed at by these pointers
- Create a new node
  - Store data in the new node
- If there are no nodes in the list, make the new node the first node
- If there are nodes in the list
  - Traverse the list to find the desired position or the end of the linked list
  - Insert the new node before the found node or at the end of the linked list

Inserting a Node into a Linked List



New node created, correct position located

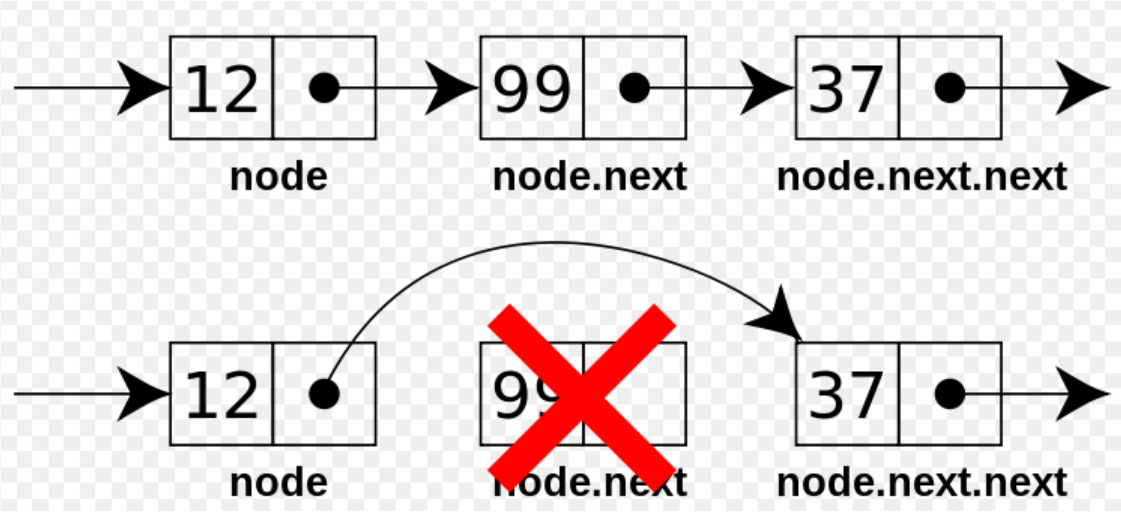
Inserting a Node into a Linked List



New node inserted in order in the linked list

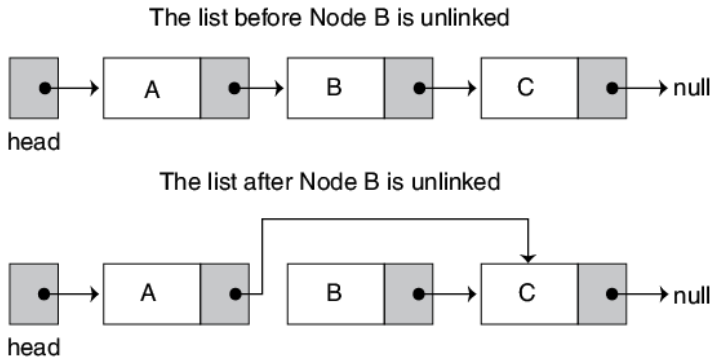
Remove From Linked List

- Pointers must be adjusted
- Memory needs to be freed



Remove

- Unlink the target node from the list
  - The node will no longer be available for traversal
- Requires two pointers
  - One to locate the node to be deleted
  - One to point to the node before the node to be deleted
- Point the previous element's next pointer to the element after the element being deleted
- Delete the target node from memory



Remove Cases

If the list is empty, then there is nothing to delete

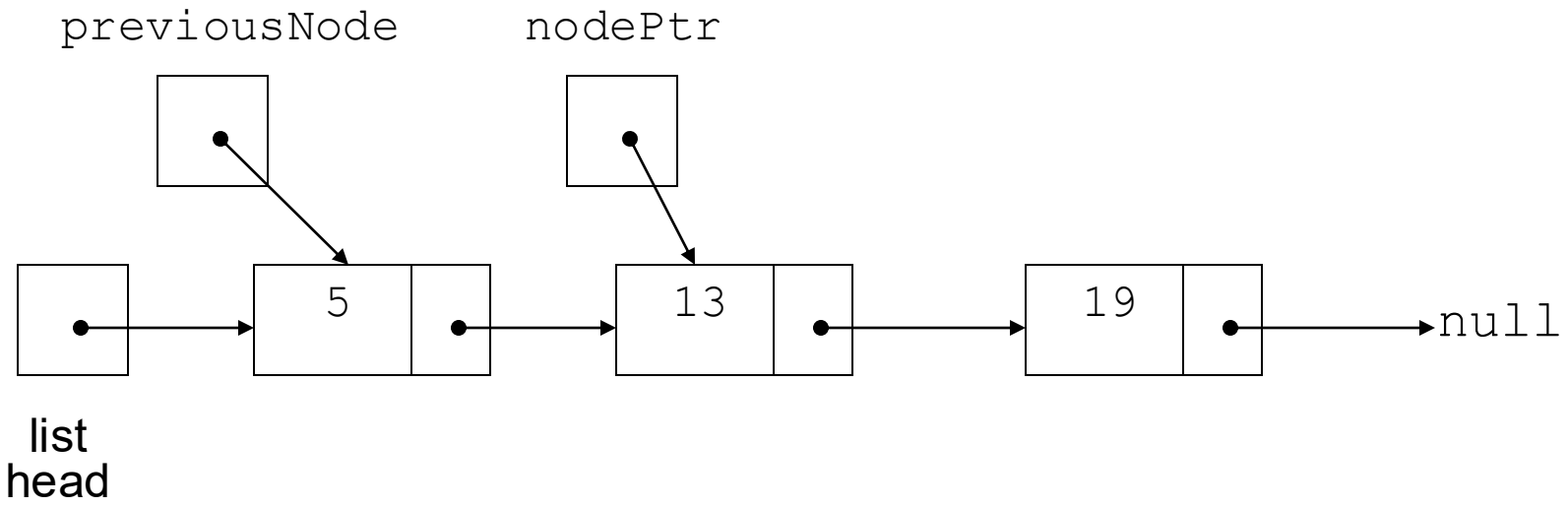
If the target node is the first node

- The head will point to the element that the target node's next pointer points to
- Delete the node

If the target is not the first element and the linked list is not empty

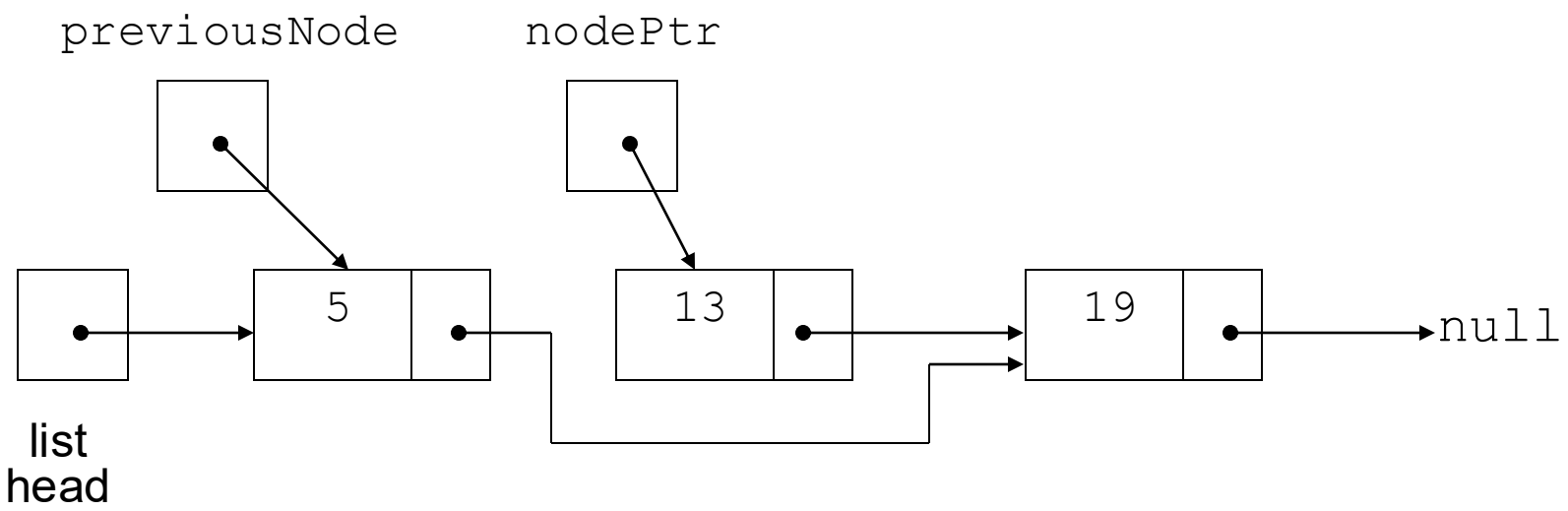
- Traverse the list until the target is found or the end of the linked list is found
  - If the end of the list is not found, set the previous node's next pointer point to the address that the target node's next pointer points to
  - Delete the target node
  - If the end of the list is reached, then there is nothing to delete – you didn't find it

Deleting a Node



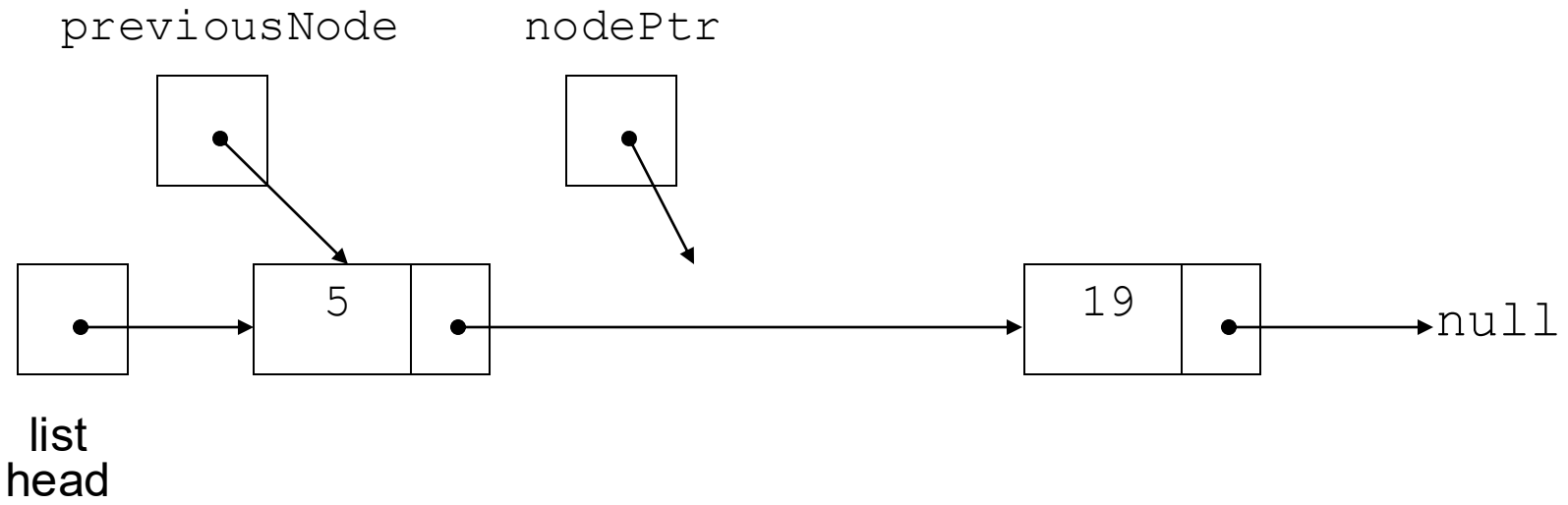
Locating the node containing `13`

Deleting a Node



Adjusting pointer around the node to be deleted

Deleting a Node



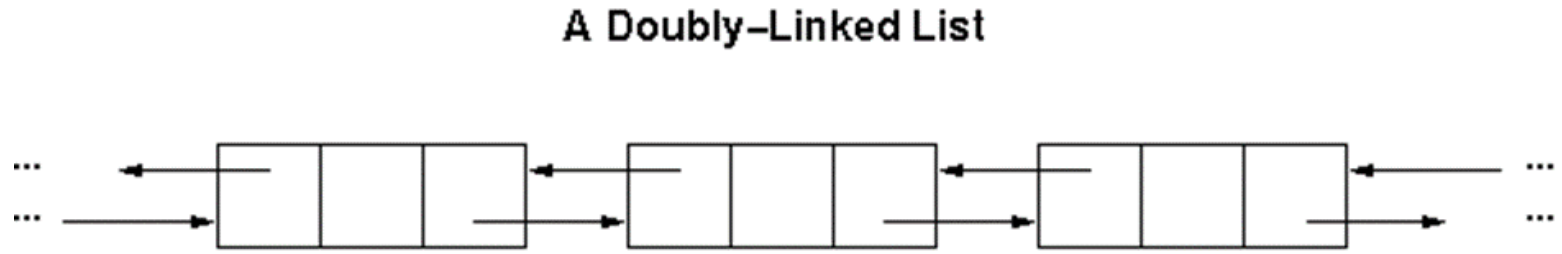
Linked list after deleting the node containing 13

Variations of Linked Lists

- Singly linked list
  - Keep a pointer to the head
  - Each node has a next pointer
  - What about keeping a pointer to the tail?
- Doubly linked list
  - Add a prev pointer to each node
- Circularly linked list
  - Last node points to the first node
- Sentinel Nodes
  - Don’t store any values
  - Dummy nodes are used to simplify implementation

Doubly Linked List

- A variation of a singly linked list
- Nodes contain two pointers
  - Prev & next



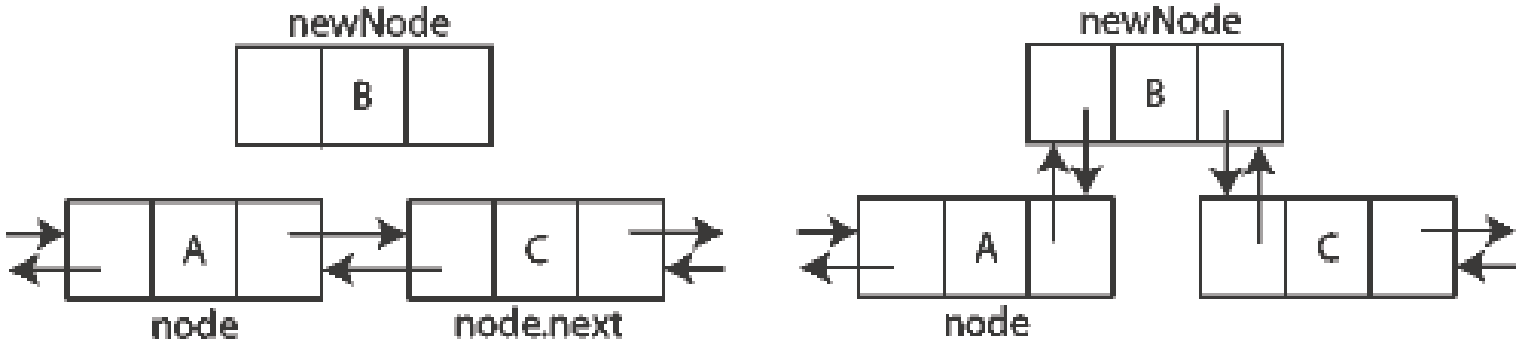
Doubly Linked List

- Link to previous and next
- Easy to traverse

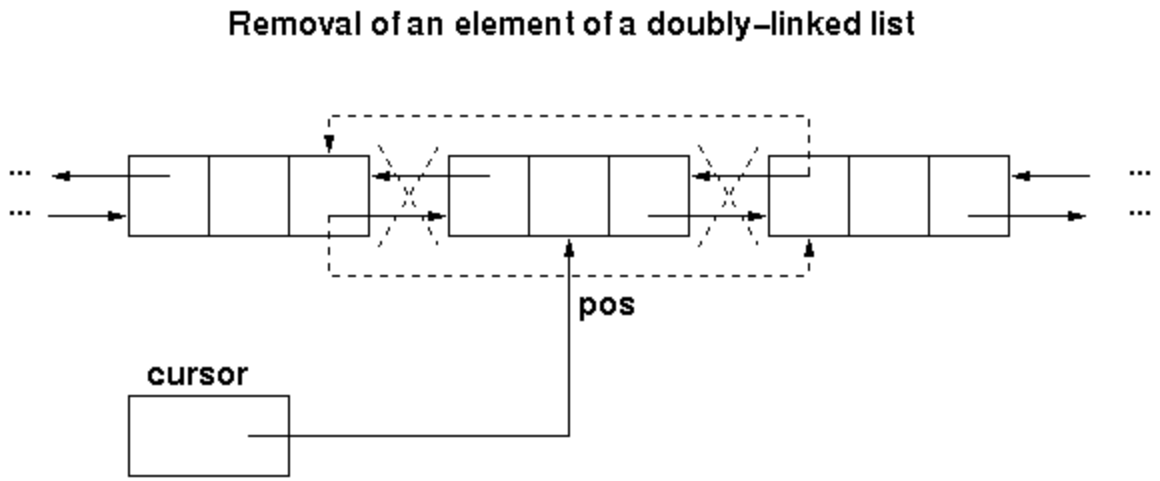
```
Struct Dlink {
        struct Dlink *next;
        struct Dlink *prev;
        TYPE val;
};
```

Insert Doubly Linked List

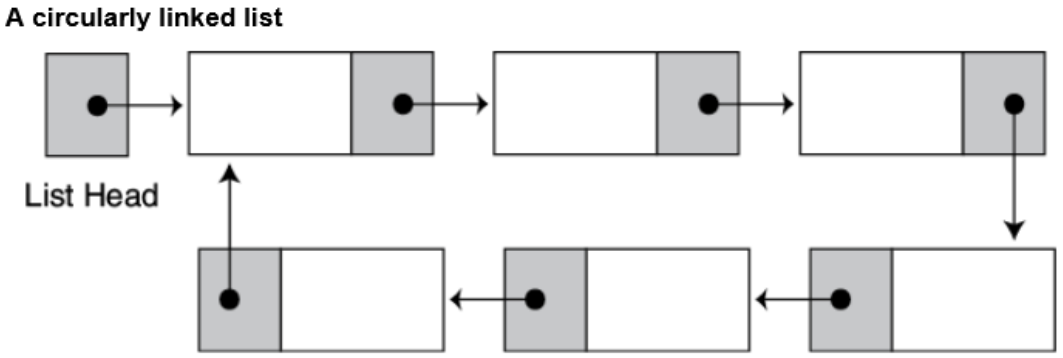
- Similar to normal list
  - Twice as many pointers



Remove Doubly Linked List

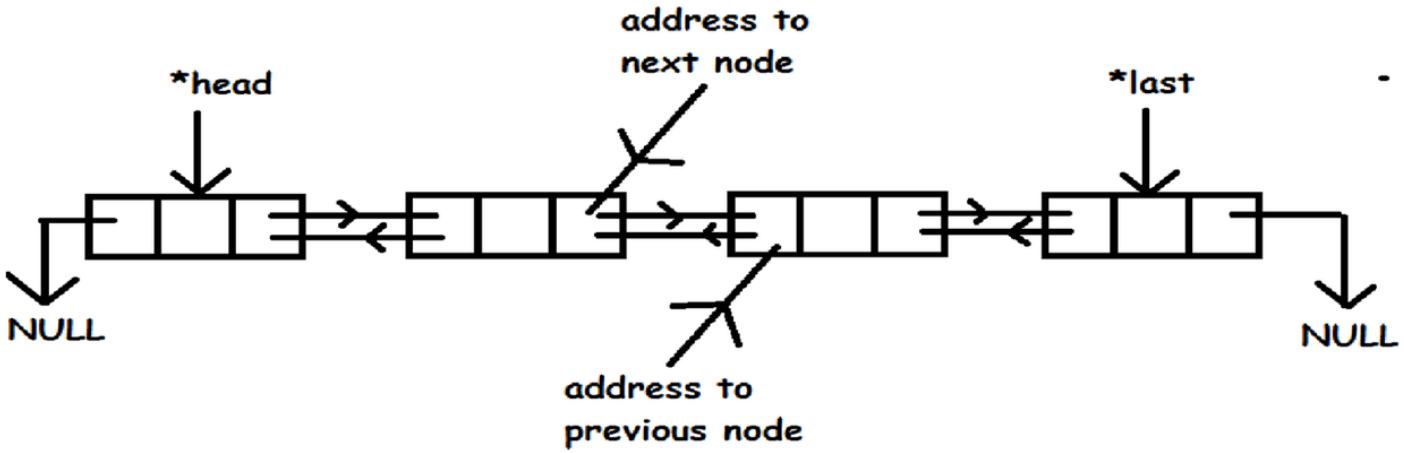


Circularly Linked List



Sentinel Links

- Used to provide quick access to the head or tail of a list.
- Many alternate names
  - Dummy, etc
- Does not store a value



Bag ADT

- Linked list implementation
  - Pros & cons vs dynamic array
- Insert
- Remove
- Search
- Display

